

SD Basic - CSV Files

Reading and writing comma-separated data without hand-rolling the quoting.

SD has statements that read and write CSV directly, and they handle the two things that make hand-written CSV code wrong: a separator inside a field, and a quote inside a field. This page covers them.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Writing

```
writcsv value1, value2, ... to file.variable then ... else ...  
printcsv {on print.unit} value1, value2, ... {:}
```

`writcsv` writes one line to a **sequential** file — see SD Basic - Sequential Files for opening one. `printcsv` sends the same thing to the terminal or a print unit.

The quoting is done for you, and it is conformant. For three values, one containing a comma and one containing quotes:

```
writcsv 'a', 'b,c', 'say "hi"' to f
```

the file receives, byte for byte:

```
a,"b,c","say ""hi"""
```

A field is quoted only when it needs to be, an embedded quote is doubled, and the result is what RFC 4180 specifies. **Do not build this by hand** — the common mistakes are quoting nothing, quoting everything, and escaping a quote with a backslash, and each produces a file that opens wrongly in a spreadsheet rather than failing visibly.

A trailing colon on `printcsv` suppresses the line terminator, so several statements can build one row.

Reading

```
readcsv from file.variable to variable1, variable2, ... then ... else ...
inputcsv variable1, variable2, ...
matreadcsv matrix from file.variable then ... else ...
```

<code>readcsv</code>	one line from a sequential file into named variables
<code>inputcsv</code>	the same, from the terminal
<code>matreadcsv</code>	one line into a dimensioned matrix, one field per element

The line written above, read back with `readcsv from f to v1, v2, v3`, gives exactly

<code>v1</code>	<code>a</code>
<code>v2</code>	<code>b,c</code>
<code>v3</code>	<code>say "hi"</code>

— a clean round trip, with the quoting removed and the embedded comma and quotes restored.

`readcsv`'s `else` branch is end of file, as `readseq`'s is.

The number of variables is fixed at compile time and the data is not. A row with more fields than you named loses the extras silently; one with fewer leaves the trailing variables empty, which is indistinguishable from a row that genuinely had empty fields. If the width varies, use `csvdq()` below and count what came back.

Splitting a line you already have

```
csvdq(line {, delimiter})
dparse.csv line, delimiter, variable1, variable2, ...
```

`csvdq()` **is a de-quoter, not a quoter. the name reads the other way.** It takes one CSV line and returns its fields separated by **field marks**, honouring the quoting:

CALL	RESULT
<code>csvdq('plain')</code>	<code>plain</code> — one field, unchanged
<code>csvdq('has,comma')</code>	<code>has</code> and <code>comma</code> — two fields
<code>csvdq('has"quote')</code>	<code>hasquote</code> — the quote was consumed as quoting
<code>csvdq('a;b', ';')</code>	<code>a</code> and <code>b</code>

So it is the function to reach for when a line arrived from somewhere else and you want it as a dynamic array:

```
fields = csvdq(line)
n = dcount(fields, @fm)
for i = 1 to n
    ...
next i
```

There is no matching function that quotes a field. To produce CSV, use `writcsv` or `printcsv`, which do it as part of writing.

`dparse.csv` does the same split but assigns straight into named variables. `dparse.csv 'x,"y,z",w', ',', p1, p2, p3` gives `x`, `y,z` and `w` — the quoted comma correctly kept inside the second field.

The separator

Every statement here defaults to a comma. `csvdq()` and `dparse.csv` take the separator as an argument; a semicolon is common in locales where the comma is the decimal separator.

A file whose separator is a semicolon is still called CSV by the program that produced it. Look at a line before assuming the separator, and take it from configuration rather than hard-coding it — the same export from the same system changes separator when the machine's locale changes.

Line endings

CSV written by SD ends each line with **CRLF** — see SD Basic - Sequential Files. That is what RFC 4180 specifies and what Windows tools expect. Reading accepts either.

What is not here

Nothing in the CSV group has been removed from this port. The whole group is **new relative to the OpenQM 2.6.6 documentation** — none of these seven names appears in its by-type list, so the old help is no guide to them.

Two behaviours were fixed in this port and are worth knowing if you are moving files across from the Linux original:

reading	a conformant CRLF file used to lose its last field on every row, because the stray CR stayed attached to it
----------------	---

writing	output is now CRLF, so a file SD produced conforms to the RFC its own documentation claims
----------------	--

See also

[SD Basic - Sequential Files](#) · [SD Basic - Data Conversion](#) · [SD Basic - String Functions](#).